

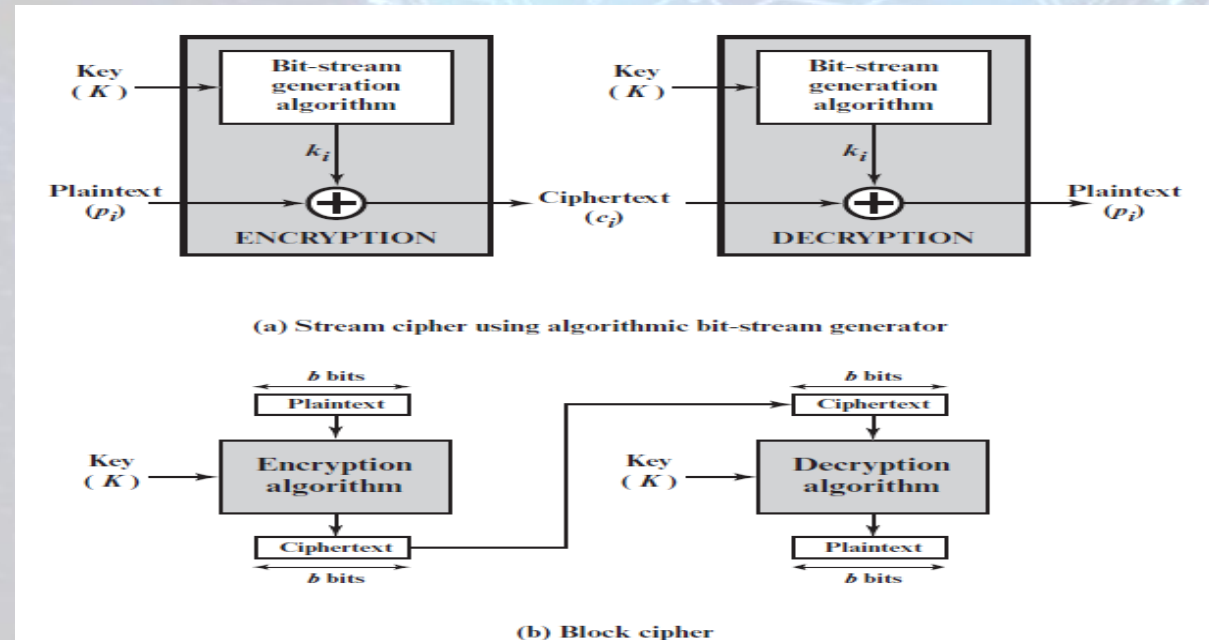


Computer Systems Security

Dr. Sandra Wahid

Stream Ciphers and Block Ciphers

- A **stream cipher** is one that encrypts a digital data stream **one bit or one byte** at a time.
- A **block cipher** is one in which a **block** of plaintext is treated as a whole and used to produce a ciphertext block of equal length.
 - Typically, a block size of 64 or 128 bits is used.
 - As with a stream cipher, the two users share a **symmetric** encryption key.
 - Applicable to a **broader range of applications** than stream ciphers.



Motivation for the Feistel Cipher Structure

- Several important symmetric block encryption algorithms in current use are based on a structure referred to as a Feistel block cipher. For that reason, it is important to examine the design principles of the Feistel cipher.
- A block cipher operates on a plaintext block of n bits to produce a ciphertext block of n bits.

→ There are 2^n possible different plaintext blocks

→ For the encryption to be reversible (i.e., for decryption to be possible), each must produce a unique ciphertext block. Such a transformation is called **reversible**, or **nonsingular**.

- So if we limit ourselves to reversible mappings, the number of different transformations is $2^n!$

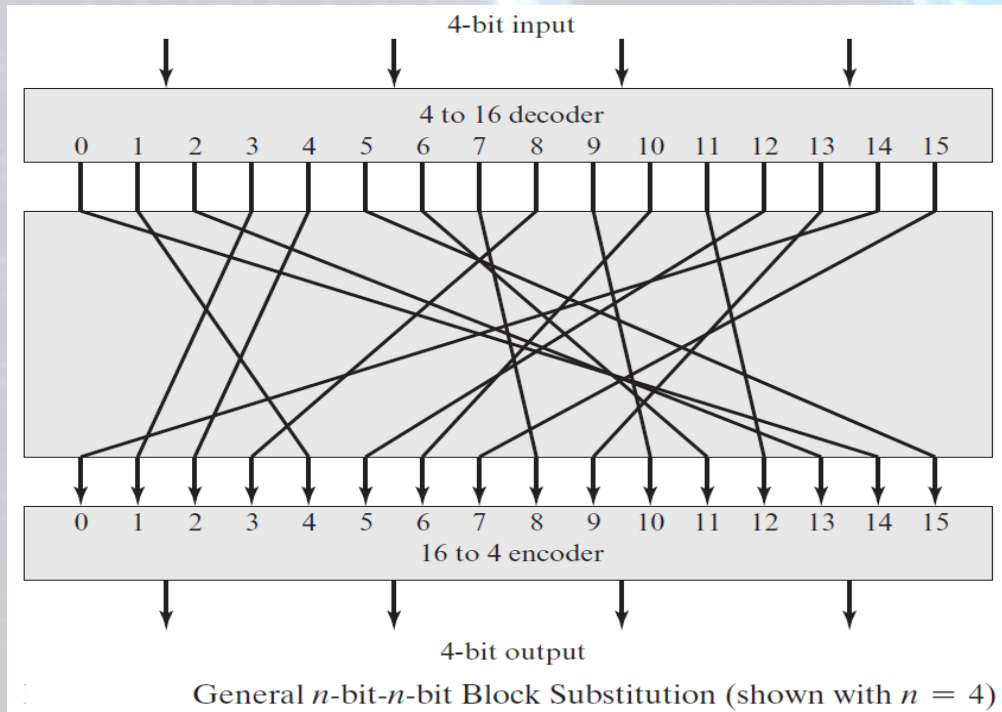
Reversible Mapping		Irreversible Mapping	
Plaintext	Ciphertext	Plaintext	Ciphertext
00	11	00	11
01	10	01	10
10	00	10	01
11	01	11	01

For the first plaintext, we can choose any of 2^n ciphertext blocks.

For the second plaintext, we choose from among $2^n - 1$ remaining ciphertext blocks, and so on.

Motivation for the Feistel Cipher Structure

- The logic of a **general substitution** cipher for $n = 4$.
 - A 4-bit input produces one of 16 possible input states, which is mapped by the substitution cipher into a unique one of 16 possible output states, each of which is represented by 4 ciphertext bits.
 - The encryption and decryption mappings can be defined by a tabulation.



Plaintext	Ciphertext	Ciphertext	Plaintext
0000	1110	0000	1110
0001	0100	0001	0011
0010	1101	0010	0100
0011	0001	0011	1000
0100	0010	0100	0001
0101	1111	0101	1100
0110	1011	0110	1010
0111	1000	0111	1111
1000	0011	1000	0111
1001	1010	1001	1101
1010	0110	1010	1001
1011	1100	1011	0110
1100	0101	1100	1011
1101	1001	1101	0010
1110	0000	1110	0000
1111	0111	1111	0101

Motivation for the Feistel Cipher Structure

- This is the most general form of block cipher and can be used to define any reversible mapping between plaintext and ciphertext.
 - Feistel refers to this as the **ideal block cipher**, because it allows for the maximum number of possible encryption mappings from the plaintext block.
- Ideal block cipher practical problems:
 1. **Small block size** → the system is equivalent to a classical substitution cipher and thus vulnerable to a statistical analysis.
 2. **Sufficiently large block size** → not practical from an implementation and performance point of view “for such a transformation, the **mapping itself constitutes the key**”.

In general, for an n -bit ideal block cipher, the length of the key defined in this fashion is $n * 2^n$ bits.

For a 64-bit block, which is a desirable length to prevent statistical attacks, the required key length is $64 * 2^{64} = 2^{70} \approx 10^{21}$ bits.

n bits	
Plaintext	Ciphertext
0000	1110
0001	0100
0010	1101
0011	0001
0100	0010
...	...
1111	0101

The Feistel Cipher

- Feistel proposed that we can **approximate the ideal block cipher** by utilizing the concept of a **product cipher**.

The essence of the approach is to develop a block cipher with a key length of *k bits* and a block length of *n bits*, allowing a total of 2^k possible transformations, rather than the $2^n!$ transformations available with the ideal block cipher.

- In particular, Feistel proposed the use of a cipher that alternates substitutions and permutations:
 - **Substitution:** Each plaintext element or group of elements is uniquely replaced by a corresponding ciphertext element or group of elements.
 - **Permutation:** A sequence of plaintext elements is replaced by a permutation of that sequence: the order in which the elements appear in the sequence is changed.

The Feistel Cipher

- In fact, Feistel's is a practical application of a proposal by Claude Shannon to develop a product cipher that alternates *confusion and diffusion* functions.
 - Shannon's concern was to prevent cryptanalysis based on **statistical analysis**.
- **Diffusion**: the statistical structure of the plaintext is dissipated into long-range statistics of the ciphertext by having **each plaintext digit affect the value of many ciphertext digits**; generally, this is equivalent to having **each ciphertext digit be affected by many plaintext digits**.
 - An example of diffusion is to encrypt a message $M = m_1, m_2, m_3, \dots$ of characters with an averaging operation:
$$y_n = \left(\sum_{i=1}^k m_{n+i} \right) \bmod 26$$
 - Diffusion can be achieved by repeatedly performing some permutation on the data followed by applying a function to that permutation; the effect is that bits from different positions in the original plaintext contribute to a single bit of ciphertext.

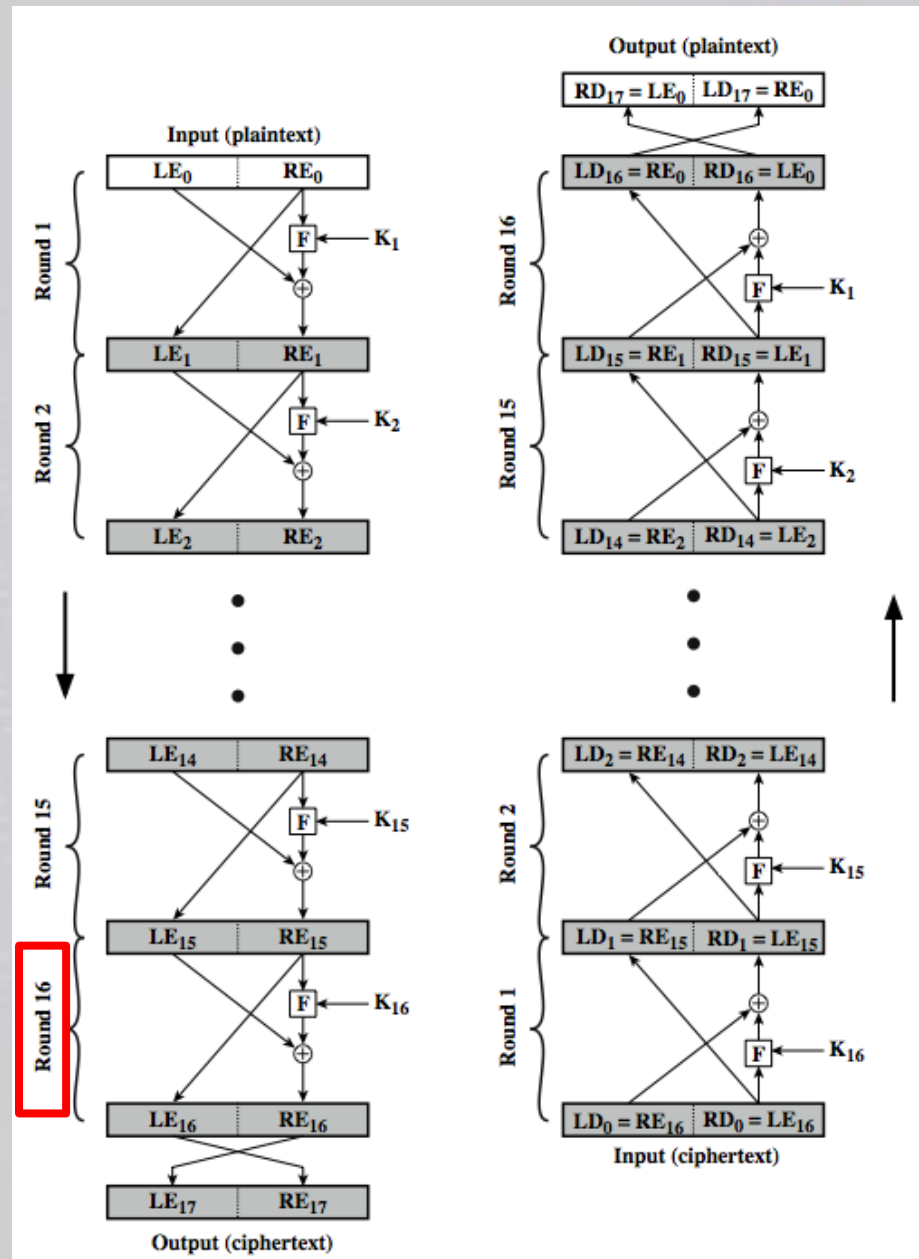
The mechanism of diffusion seeks to make the **statistical relationship between the plaintext and ciphertext as complex as possible** in order to prevent attempts to deduce the key.

The Feistel Cipher

- **Confusion:** seeks to make the **relationship between the statistics of the ciphertext and the value of the encryption key as complex as possible**, again to prevent attempts to discover the key.

So successful are diffusion and confusion in capturing the essence of the desired attributes of a block cipher that they have become the **cornerstone** of modern block cipher design.

Feistel Cipher Structure



- The inputs to the encryption algorithm are a plaintext block of length **$2w$** bits and a key **K** .
- The plaintext block is divided into two halves: **LE** and **RE**
- The two halves of the data pass through **n rounds**.
- Each round **i** has as inputs **LE_{i-1}** and **RE_{i-1}** derived from the previous round, as well as a subkey **K_i** derived from the overall **K**
 → In general, the subkeys **K_i** are different from **K** and from each other
- All rounds have the same structure.
- A **substitution** is performed on the **left half** of the data.
- Following this substitution, a **permutation** is performed that consists of the interchange of the two halves of the data.
- The process of **decryption** is essentially the **same as the encryption**.
- The rule is: use the ciphertext as input to the algorithm, but use the subkeys **K_i** in **reverse order**.

• Encryption Algorithm Round i

$$L_i = R_{i-1}$$

$$R_i = L_{i-1} \oplus F(R_{i-1}, K_i)$$

Round function F

Feistel Cipher Structure

How decryption works?

- After the last iteration of the encryption process, the two halves of the output are swapped, so that the ciphertext is $RE_{16} || LE_{16}$

$$LE_{16} = RE_{15}$$

$$RE_{16} = LE_{15} \oplus F(RE_{15}, K_{16})$$

$$LD_1 = RD_0 = LE_{16} = RE_{15}$$

$$RD_1 = LD_0 \oplus F(RD_0, K_{16})$$

$$= RE_{16} \oplus F(RE_{15}, K_{16})$$

$$= [LE_{15} \oplus F(RE_{15}, K_{16})] \oplus F(RE_{15}, K_{16})$$

The XOR has the following properties:

$$[A \oplus B] \oplus C = A \oplus [B \oplus C]$$

$$D \oplus D = 0$$

$$E \oplus 0 = E$$

$$RD_1 = LE_{15}$$

For the i th iteration of the encryption algorithm,

$$LE_i = RE_{i-1}$$

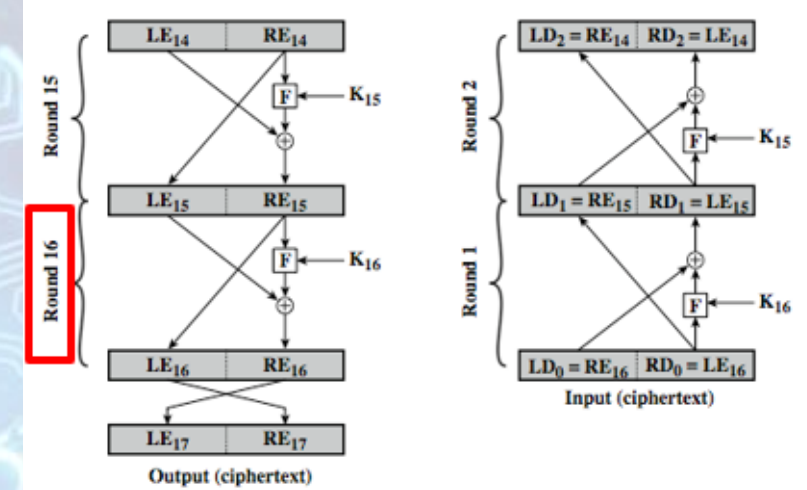
$$RE_i = LE_{i-1} \oplus F(RE_{i-1}, K_i)$$

Rearranging terms:

$$RE_{i-1} = LE_i$$

$$LE_{i-1} = RE_i \oplus F(RE_{i-1}, K_i) = RE_i \oplus F(LE_i, K_i)$$

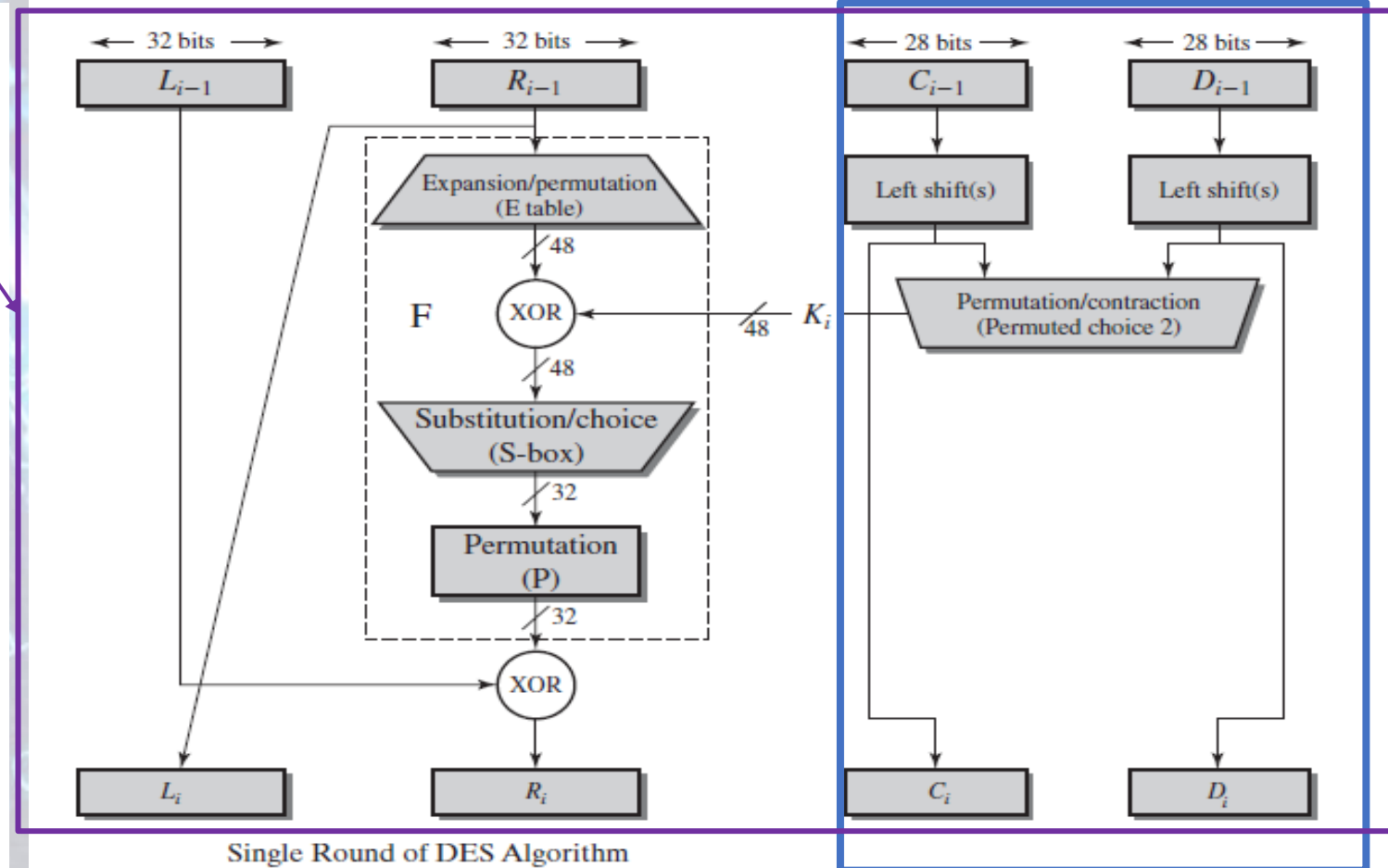
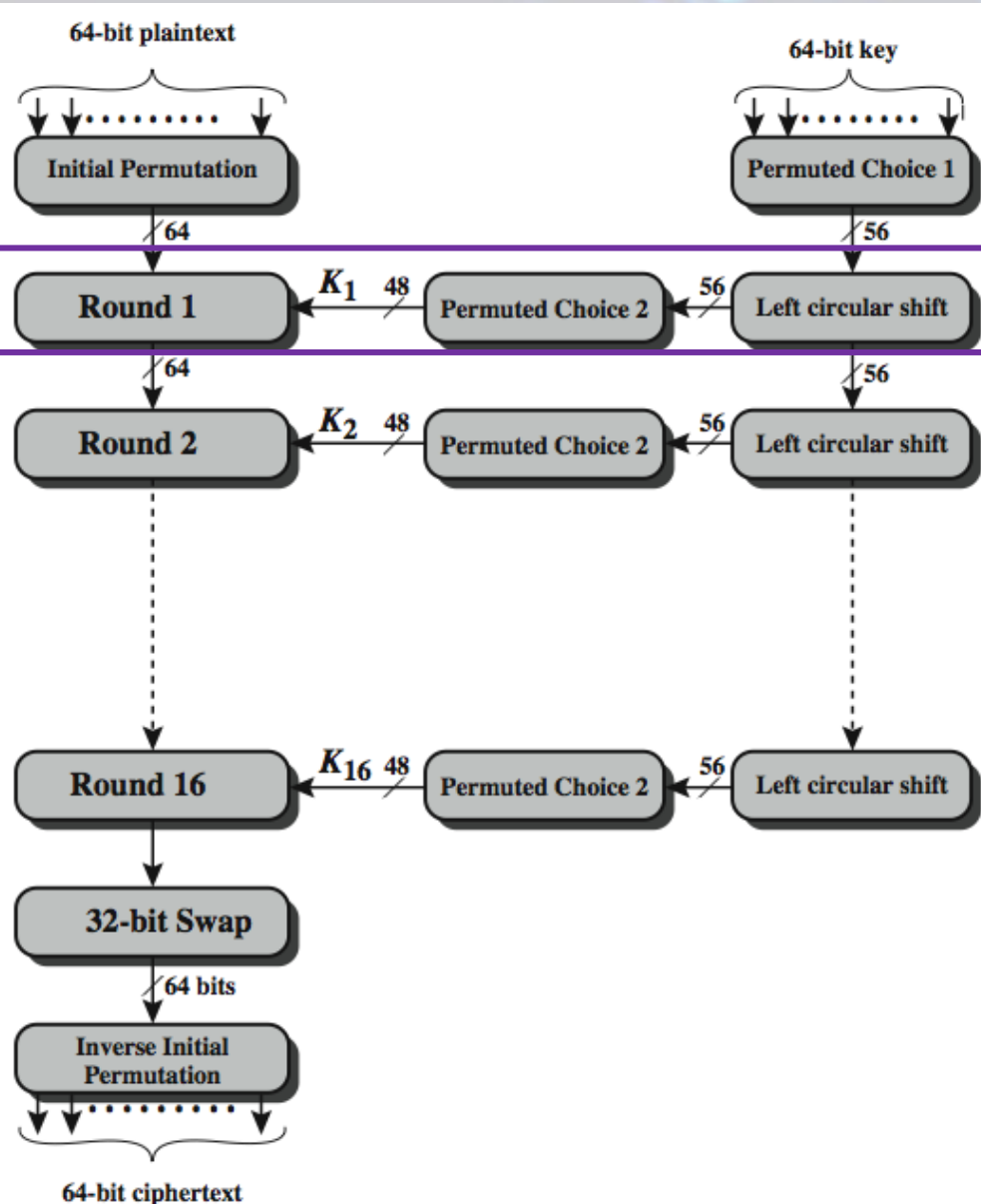
- The output of the last round of the decryption process is $RE_0 || LE_0$ after a swap $\rightarrow$ plaintext



The Data Encryption Standard (DES)

- The **plaintext** must be **64 bits** in length and the **key** is **56 bits** in length.

For sub-keys generation



- With the exception of the initial and final permutations, DES has the exact structure of a Feistel cipher.
- As with any Feistel cipher, decryption uses the same algorithm as encryption, except that the application of the subkeys is reversed. Additionally, the initial and final permutations are reversed.

The Data Encryption Standard (DES)

Initial and Inverse Initial Permutations:

- The initial permutation and its inverse are defined by tables.
- The 64 entries in the permutation table contain a permutation of the numbers from 1 to 64.

(a) Initial Permutation (IP)							
58	50	42	34	26	18	10	2
60	52	44	36	28	20	12	4
62	54	46	38	30	22	14	6
64	56	48	40	32	24	16	8
57	49	41	33	25	17	9	1
59	51	43	35	27	19	11	3
61	53	45	37	29	21	13	5
63	55	47	39	31	23	15	7

(b) Inverse Initial Permutation (IP^{-1})							
40	8	48	16	56	24	64	32
39	7	47	15	55	23	63	31
38	6	46	14	54	22	62	30
37	5	45	13	53	21	61	29
36	4	44	12	52	20	60	28
35	3	43	11	51	19	59	27
34	2	42	10	50	18	58	26
33	1	41	9	49	17	57	25

Input size= 64 bits
Output size= 64 bits

Input size= 64 bits
Output size= 64 bits

The Data Encryption Standard (DES)

Expansion Permutation (E):

32	1	2	3	4	5
4	5	6	7	8	9
8	9	10	11	12	13
12	13	14	15	16	17
16	17	18	19	20	21
20	21	22	23	24	25
24	25	26	27	28	29
28	29	30	31	32	1

Input size= 32 bits
Output size= 48 bits

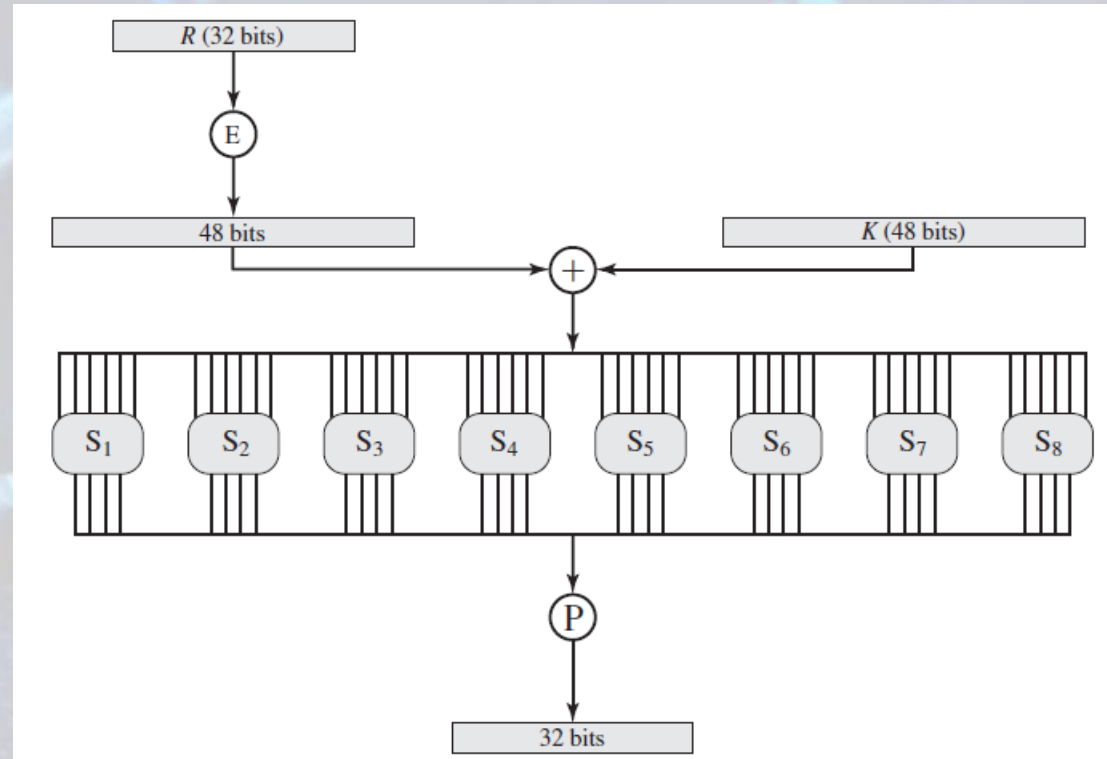
Permutation Function (P):

16	7	20	21	29	12	28	17
1	15	23	26	5	18	31	10
2	8	24	14	32	27	3	9
19	13	30	6	22	11	4	25

Input size= 32 bits
Output size= 32 bits

The Data Encryption Standard (DES)

Calculation of $F(R, K)$:



S-boxes:

- The substitution consists of a set of **eight S-boxes**, each of which **accepts 6 bits (48/8)** as input and **produces 4 bits** as output.

The Data Encryption Standard (DES)

S-boxes:

S_1	14	4	13	1	2	15	11	8	3	10	6	12	5	9	0	7
	0	15	7	4	14	2	13	1	10	6	12	11	9	5	3	8
	4	1	14	8	13	6	2	11	15	12	9	7	3	10	5	0
	15	12	8	2	4	9	1	7	5	11	3	14	10	0	6	13
S_2	15	1	8	14	6	11	3	4	9	7	2	13	12	0	5	10
	3	13	4	7	15	2	8	14	12	0	1	10	6	9	11	5
	0	14	7	11	10	4	13	1	5	8	12	6	9	3	2	15
	13	8	10	1	3	15	4	2	11	6	7	12	0	5	14	9

-Values in the cells in the range [0-15]
since the output should be 4 bits
-with repetition since the total
number of cells is $4 \times 16 = 64$

- The first and last bits of the input to box S_i form a 2-bit binary number to select one of four substitutions defined by the four rows in the table for S_i .
- The middle four bits select one of the sixteen columns.
- The decimal value in the cell selected by the row and column is then converted to its 4-bit representation to produce the output.
- Rows and columns counting begin by zero.
- For example, in S_1 , for input **011001**, the row is **01 (row 1)** and the column is **1100 (column 12)** → cell has 9, so the output is 1001.

The Data Encryption Standard (DES)

Key Generation:

- A 64-bit key is used as input to the algorithm, the bits of the key are numbered from 1 through 64; every eighth bit is ignored:

(a) Input Key

1	2	3	4	5	6	7	8
9	10	11	12	13	14	15	16
17	18	19	20	21	22	23	24
25	26	27	28	29	30	31	32
33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48
49	50	51	52	53	54	55	56
57	58	59	60	61	62	63	64

Input size= 64 bits
Output size= 56 bits

- The key is first subjected to a permutation governed by a table labeled **Permuted Choice One**

(b) Permuted Choice One (PC-1)

57	49	41	33	25	17	9
1	58	50	42	34	26	18
10	2	59	51	43	35	27
19	11	3	60	52	44	36
63	55	47	39	31	23	15
7	62	54	46	38	30	22
14	6	61	53	45	37	29
21	13	5	28	20	12	4

Input size= 56 bits
Output size= 56 bits

The Data Encryption Standard (DES)

Key Generation:

- The resulting 56-bit key is then treated as two **28-bit quantities**: C_0 and D_0
- At each round, C_{i-1} and D_{i-1} are separately subjected to a circular left shift or (rotation) of 1 or 2 bits, as governed by:

(d) Schedule of Left Shifts

Round Number	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Bits Rotated	1	1	2	2	2	2	2	2	1	2	2	2	2	2	2	1

- These shifted values serve as input to the next round.
- They also serve as input to the part labeled **Permuted Choice Two** which produces a 48-bit output that serves as input key to the function F.

(c) Permuted Choice Two (PC-2)

14	17	11	24	1	5	3	28
15	6	21	10	23	19	12	4
26	8	16	7	27	20	13	2
41	52	31	37	47	55	30	40
51	45	33	48	44	49	39	56
34	53	46	42	50	36	29	32

Input size= 56 bits
Output size= 48 bits

Not all the 56 numbers
are found → table is
 $8 \times 6 = 48$

The Avalanche Effect

- A desirable property of any encryption algorithm is that **a small change in either the plaintext or the key should produce a significant change in the ciphertext.**
- In particular, a change in one bit of the plaintext or one bit of the key should produce a change in many bits of the ciphertext.
- Avalanche Effect in DES: Change in Plaintext

Round		δ
	02468aceeca86420 12468aceeca86420	1
1	3cf03c0fbad22845 3cf03c0fbad32845	1
2	bad2284599e9b723 bad3284539a9b7a3	5
3	99e9b7230bae3b9e 39a9b7a3171cb8b3	18
4	0bae3b9e42415649 171cb8b3ccaca55e	34
5	4241564918b3fa41 ccaca55ed16c3653	37
6	18b3fa419616fe23 d16c3653cf402c68	33
7	9616fe2367117cf2 cf402c682b2cefbc	32
8	67117cf2c11bfc09 2b2cefbc99f91153	33

Round		δ
9	c11bfc09887fbc6c 99f911532eed7d94	32
10	887fbc6c600f7e8b 2eed7d94d0f23094	34
11	600f7e8bf596506e d0f23094455da9c4	37
12	f596506e738538b8 455da9c47f6e3cf3	31
13	738538b8c6a62c4e 7f6e3cf34bc1a8d9	29
14	c6a62c4e56b0bd75 4bc1a8d91e07d409	33
15	56b0bd7575e8fd8f 1e07d4091ce2e6dc	31
16	75e8fd8f25896490 1ce2e6dc365e5f59	32
IP^{-1}	da02ce3a89ecac3b 057cde97d7683f2a	32

- The second column of the table shows the intermediate 64-bit values at the end of each round for two plaintexts.
- The third column shows the number of bits that differ between the two intermediate values.
- The table shows that, after just three rounds, 18 bits differ between the two blocks which were initially different in 1 bit only.
- On completion, the two ciphertexts differ in 32 bit positions.

The Avalanche Effect

- Avalanche Effect in DES: Change in Key

Round		δ
	02468aceeca86420 02468aceeca86420	0
1	3cf03c0fbad22845 3cf03c0f9ad628c5	3
2	bad2284599e9b723 9ad628c59939136b	11
3	99e9b7230bae3b9e 9939136b768067b7	25
4	0bae3b9e42415649 768067b75a8807c5	29
5	4241564918b3fa41 5a8807c5488dbe94	26
6	18b3fa419616fe23 488dbe94aba7fe53	26
7	9616fe2367117cf2 aba7fe53177d21e4	27
8	67117cf2c11bfc09 177d21e4548f1de4	32

Round		δ
9	c11bfc09887fbc6c 548f1de471f64dfd	34
10	887fbc6c600f7e8b 71f64dfd4279876c	36
11	600f7e8bf596506e 4279876c399fdc0d	32
12	f596506e738538b8 399fdc0d6d208dbb	28
13	738538b8c6a62c4e 6d208dbbb9bdeea	33
14	c6a62c4e56b0bd75 b9bdeeaad2c3a56f	30
15	56b0bd7575e8fd8f d2c3a56f2765c1fb	27
16	75e8fd8f25896490 2765c1fb01263dc4	30
IP^{-1}	da02ce3a89ecac3b ee92b50606b62b0b	30

- The table shows a similar test using the original plaintext with two keys that differ in only 1 bit position.
- Again, the results show that about half of the bits in the ciphertext differ and that the avalanche effect is pronounced after just a few rounds.

The Strength of DES

The Use of 56-Bit Keys:

Table shows how much time is required for a brute-force attack for various key sizes.

Key Size (bits)	Cipher	Number of Alternative Keys	Time Required at 10^9 Decryptions/s	Time Required at 10^{13} Decryptions/s
56	DES	$2^{56} \approx 7.2 \times 10^{16}$	$2^{55} \text{ ns} = 1.125 \text{ years}$	1 hour
128	AES	$2^{128} \approx 3.4 \times 10^{38}$	$2^{127} \text{ ns} = 5.3 \times 10^{21} \text{ years}$	$5.3 \times 10^{17} \text{ years}$
168	Triple DES	$2^{168} \approx 3.7 \times 10^{50}$	$2^{167} \text{ ns} = 5.8 \times 10^{33} \text{ years}$	$5.8 \times 10^{29} \text{ years}$
192	AES	$2^{192} \approx 6.3 \times 10^{57}$	$2^{191} \text{ ns} = 9.8 \times 10^{40} \text{ years}$	$9.8 \times 10^{36} \text{ years}$
256	AES	$2^{256} \approx 1.2 \times 10^{77}$	$2^{255} \text{ ns} = 1.8 \times 10^{60} \text{ years}$	$1.8 \times 10^{56} \text{ years}$
26 characters (permutation)	Monoalphabetic	$26! = 4 \times 10^{26}$	$2 \times 10^{26} \text{ ns} = 6.3 \times 10^9 \text{ years}$	$6.3 \times 10^6 \text{ years}$

Fortunately, there are a number of alternatives to DES, the most important of which are AES (Advanced Encryption Standard) and triple DES

The Strength of DES

Timing Attacks:

- Timing attack is one in which information about the key or the plaintext is obtained by observing **how long** it takes a given implementation to perform decryptions on various ciphertexts.
- A timing attack exploits the fact that an encryption or decryption algorithm often takes slightly different amounts of time on different inputs.
- **DES appears to be fairly resistant** to a successful timing attack.

Block Cipher Design Principles

Number of Rounds:

- The **greater the number** of rounds, the more difficult it is to perform cryptanalysis, even for a relatively weak F.

Design of Function F:

- F provides the element of **confusion** in a Feistel cipher. Thus, it must be difficult to “unscramble” the substitution performed by F.
- One obvious criterion is that F be nonlinear → the more nonlinear F, the more difficult any type of cryptanalysis will be.

Key Schedule Algorithm:

- With any Feistel block cipher, the key is used to generate one subkey for each round.
- In general, we would like to select subkeys to maximize the difficulty of deducing individual subkeys and the difficulty of working back to the main key.
- No general principles for this yet.

A digital padlock with a circuit board design, set against a background of binary code. The padlock is metallic and has a glowing blue circuit pattern on its body. The background is dark blue with white binary digits (0s and 1s) scattered across it.

Thank You